

Analysis: Festival

Test Set 1

We can consider each attraction as an interval that covers some range of days and has some *happiness rating*. Let us denote *happiness rating* of an interval as HR.

At first we sort the intervals according to their HRs, from higher to lower. Then we will iterate through each day and consider that day as the day we will attend the festival. Let us denote this day as d_i . And let us denote the total happiness of upto K intervals on this day as c_i .

Let us set c_i to 0. Now we will iterate through the sorted intervals and check if d_i is included in the current interval. If yes, then we add the HR of that interval to c_i . If no, we continue to the next interval. If the number of intervals taken into account reaches K , we stop calculating c_i and compare it to the current maximum answer and store it as maximum if it is greater. If we iterate through all the intervals and the number of intervals taken is less than K , we still compare c_i to the current maximum answer and store it as maximum if it is greater.

The answer is the maximum among all c_i s.

The time complexity of this solution is $O(N \times \log N + D \times N)$.

The space complexity (without considering the input) of this solution is $O(1)$ as no extra memory is used.

A pseudocode would look something like this:

```
ans = 0
sort(intervals) // from higher to lower
for d_i = 1 to D:
    c_i = 0
    cnt = 0
    for j = 1 to N:
        if (cnt == K): break
        if (s_j <= d_i <= e_j):
            c_i += h[d_i]
            cnt++
    ans = max(ans, c_i)
return ans
```

Test Set 2

Now the brute force will not work. We can still iterate through each day, but now we cannot check each interval to see if this day is in that interval or not. Rather for each day, we will update the events taking place on that day efficiently and then calculate the total HR for that day. We will consider the ending days as the $(e_i + 1)$ -th days, because the e_i -th day is included in the interval. So now we will consider the $(e_i + 1)$ -th day as the ending day for each interval. For example, if an interval ended on day 6 previously, now we will consider that it ends on day 7.

For each day, we will store the intervals that either start or end on that day.

We will use segment trees for our solution. Each leaf node of the tree will be an interval and the intervals will be sorted according to their HRs, from higher to lower. Initially every interval will have a value of 0. When an interval becomes active, we will update that index with its HR. And when an interval becomes inactive, we will update that index with 0 again. We need to query the sum of any range in the trees. We also need another operation. At any moment we need to find the index of the K -th maximum (HR wise) active interval. If there are less than K active intervals, we will return the index of the rightmost/last active interval. We can use another segment tree for this.

Now we will iterate through the days.

If the current day is a starting day for an interval, it means this interval has become active. We will update its index with its HR in one tree and will make it active in the other one.

If the current day is an ending day for an interval, it means this interval has become inactive. We will update its index with 0 in one tree and will make it inactive in the other one.

We will do this for each interval for which the current day is a starting day or an ending day.

On the days that we update the segment trees, after updating, we will query for the index of the K -th maximum active interval in the second tree. Let us assume this index is idx . And then in the first tree, we will query the sum from the 0-th index to index idx . We will store this sum in the current answer if the current answer is smaller.

The answer is the final value in the current answer.

The time complexity of building a segment tree with n elements is $O(n)$ and for updating and querying the time complexity is $O(\log n)$ for each operation.

So the time complexity of this solution is $O(N + D + N \times \log N)$.

And the space complexity of this solution is $O(N)$.

Alternative Solution

We will discuss another solution using [multisets](#) which is available in C++ standard template library (STL). It can also be implemented using a [binary search tree](#) in any language.

We will use two multisets, one to keep track of the K active intervals with higher HRs, and the other one to keep track of the rest of the active intervals.

Now we will iterate through the days again.

If the current day is a starting day of an interval, then if the number of active intervals in the first multiset is less than K , we insert this interval in the first multiset. Otherwise, if the interval with the smallest HR has a greater HR than this interval, then we insert this interval in the second multiset. And if it is less or equal, then we remove the smallest from the first multiset and insert the removed interval into the second multiset. And then we insert the current interval in the first multiset.

If the current day is an ending day of an interval, then if this interval is in the second multiset, we simply remove it from there. Otherwise it is in the first multiset. So, we remove the current interval from the first multiset. Then we remove the interval with the largest HR from the second multiset and insert the removed interval in the first multiset.

We always maintain a variable *sum* which is the sum of the HRs of the intervals in the first multiset. While inserting/removing an interval in/from the first multiset, we have to update the value of the *sum* accordingly.

The answer is the maximum value of *sum* across the whole time.

The time complexity of accessing, inserting, removing elements from a multiset with n elements is $O(\log n)$ for each operation. Since there are $\mathbf{N}$ intervals and each interval will be handled 2 times and each handling will require $O(1)$ operations with the multisets, the overall complexity of all operations with multisets will be $O(\mathbf{N} \times \log \mathbf{N})$.

So, the overall complexity of this solution is: $O(\mathbf{D} + \mathbf{N} \times \log \mathbf{N})$.

And the space complexity of this solution is $O(\mathbf{N})$.

Analysis: Graph Travel

Test Set 1

We use a brute force strategy by DFS:

1. Pick a room to start. Add it to the path and save the path as visited. If we have $\mathbf{K}$ points, increment our paths counter.
2. Pick a room outside the path that keeps our total points under $\mathbf{K}$. Check if any corridor connects the room to another room in the path. If yes, add the new connected room to the path and remove the corridor from the set. If no, try the next room outside the path. Evaluating each room choice requires checking every corridor and takes $O(\mathbf{M})$ time.
3. If we are out of corridors or our total points $\geq \mathbf{K}$, we backtrack. Remove the latest room from the path, and try the next available corridor.

In terms of run time in the worst case, we try each possible ordering of rooms. There are $\mathbf{N}!$ orderings. Each ordering has $\mathbf{N}$ room "addition" steps and each step takes us $\mathbf{M}$ time to evaluate (see step 2 above). That gives us $O(\mathbf{N}! \times \mathbf{M})$

Test Set 2

Brute force is not good enough. We approach this problem with dynamic programming. Our strategy will be the following:

1. Find the total magic points of every combination of rooms regardless of whether they can be visited.
2. For each combination of rooms, determine if a room can be visited as a new room.
3. Use dynamic programming to determine which combinations can actually be visited given any single room as a starting point.
4. Filter down to room combinations with point total $\mathbf{K}$ that can be visited.

There are $2^{\mathbf{N}}$ different ways to pick combinations of rooms. Let us first assume that it is possible to visit that combination of rooms and calculate the total magic points. Since there are $2^{\mathbf{N}}$ possible combinations, we can express each combination as a number whose binary representation tells us whether we visit that room. For example:

- $7 = 111$ contains rooms 1, 2, and 3
- $4 = 100$ contains only room 3

We can express the total number of magic points for each room combination as a list of length $2^{\mathbf{N}}$ and takes $O(2^{\mathbf{N}} \times \mathbf{N})$ time to compute. We can also do it in $O(2^{\mathbf{N}})$ using Sum Over Subset Dp technique.

Next, assume we have just visited a combination of rooms, we want to find out if we are able to visit a particular room and add it as a new unique room.

1. If at least one of the rooms in starting combination is adjacent to the new room. To check this, we can maintain a bitmask of rooms adjacent to each room (adjacency bitmask) and then we can check if the starting combination and adjacency bitmask have a common element (this can be done checking that the AND of starting combination and adjacency bitmask is non zero).

2. We have the correct number of points to break that room's shield (say our destination is room i and we have P points, this means $\mathbf{L}_i \leq P \leq \mathbf{R}_i$).

We can express this information as a $2^{\mathbf{N}} \times \mathbf{N}$ list of lists called `canVisit`. Let us say we find that we can visit room i from combination of rooms c . `canVisit[c][i] = true`. We also allow ourselves to start in any room so `canVisit[0][i] = true`. This list of lists takes $O(2^{\mathbf{N}} \times \mathbf{N} + \mathbf{M})$ time to compute since checking each of above two steps takes $O(1)$ time and the computing of adjacency bitmasks takes $O(\mathbf{M})$ time because we need to iterate over all the corridors.

Now we use dynamic programming. We iterate through all the integers between 0 and $2^{\mathbf{N}}$, each integer representing a possible combination of rooms. For a combination x , we iterate through every room already in x . We remove a room and check if it is possible to visit that room (i) from the remaining combination (x') using the `canVisit` array (`canVisit[x'][i]`). The number of ways to visit combination x is equal to the sum of all the numbers of ways to visit all possible x' . Since we are iterating from smaller combinations up, we know we only need to iterate once. This will tell us which room combinations can be visited. This takes $O(2^{\mathbf{N}} \times \mathbf{N} + \mathbf{M})$

Finally, we pick out all the room combinations that have total of $\mathbf{K}$ points. We filter that down to the combinations that we can actually visit and sum all the unique ways to visit that combination. This just takes $O(2^{\mathbf{N}})$ time. Our final run time is $O(2^{\mathbf{N}} \times \mathbf{N} + \mathbf{M})$

Analysis: Star Trappers

Test Set 1

Say, we have just one polygon and the point we need to capture (called P henceforth). We need a way to check if that point is inside the polygon or not. This problem is known as [Point in Polygon](#) and ray casting is one of the standard ways to solve it. Now, for the first test set, we can generate all possible simple polygons and check the minimum perimeter polygon which contains the point. Following steps show how to generate all simple polygons:

1. Generate all distinct permutations from the given set of points (minimum 3 points). Treat each permutation as a polygon with points describing consecutive vertices of that polygon.
2. For each polygon, if any two edges intersect each other, discard that polygon.
3. All remaining permutations will describe all possible simple polygons.

Runtime Analysis: To generate R distinct permutations from N points will take N [permutations](#) R (called P_R^N) and then checking for intersection of any two edges will take $(R - 1) \times (R - 2)$. Then for each polygon it will take $(R - 1)$ checks for ray casting. So, total runtime will be:

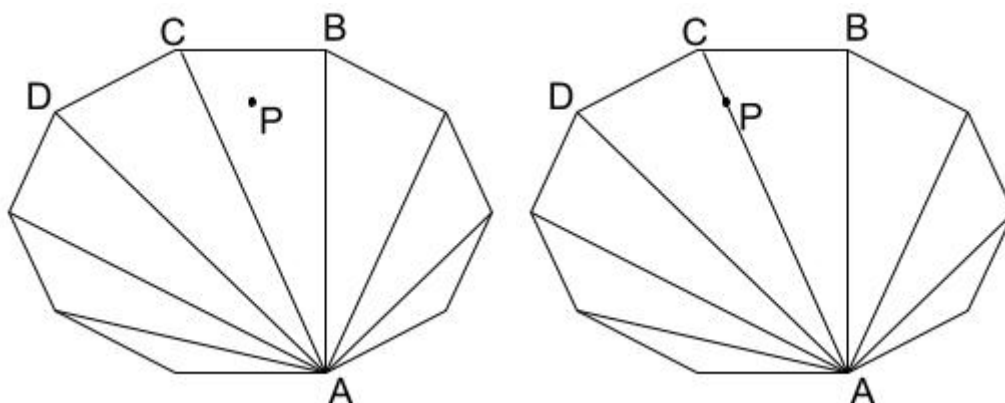
$$O(\sum_{R=3}^N P_R^N \times (R - 1) \times (R - 2) \times (R - 1)) = O(\sum_{R=3}^N P_R^N \times R^3) = O(N! \times N^3).$$

Note: This might look like it will TLE but most of the checks would not be performed. We can improve this runtime, though, using [convex hull](#). A convex hull of a simple polygon will always include the initial area of a simple polygon and will have smaller or equal perimeter. The runtime in that case will be:

$$O(\sum_{R=3}^N C_R^N \times (R \times \log(R)) \times (R - 1)) = O(2^N \times N^2 \times \log(N))$$

Test Set 2

Notice that if a point is inside a polygon with more than 4 points, we can reduce it to a triangle (ABC in the image below) or a quadrilateral ($ABCD$ in the image below) which contains the point.



Note: It can be some other triangle/quadrilateral in the above polygon, the image is just for illustration.

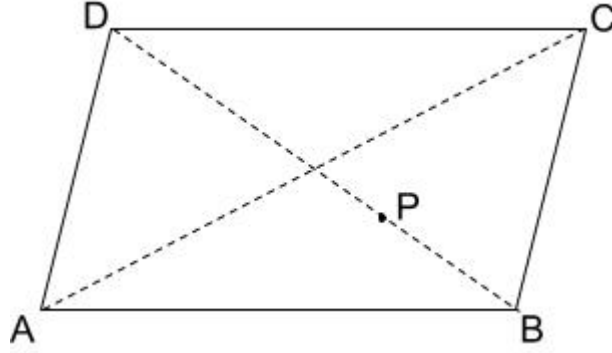
So, we can generate all possible quadrilaterals and triangles and then check the minimum perimeter polygon which contains the point.

Runtime Analysis: We can generate all sets of points with set size being 4 (for quadrilateral) and 3 (for triangle) from the given set of points taking $O(N^4 + N^3) = O(N^4)$. Checking "Point in

Polygon" in this case will be constant time.

Test Set 3

The observation required for this set is that if the point P is on one of the diagonals in the quadrilateral, it must be at the intersection of the diagonals. If it is not, then we can reduce the quadrilateral to a triangle with a lesser perimeter (as in the following diagram, quadrilateral $ABCD$ can be reduced to triangle ABC).



Another observation is that among the points collinear to P only one point closest to P on both side matters. Other points will always create quadrilateral with larger perimeter. So, all collinear segments of interest are unique.

So, for this set we can check all the triangles first. Then to generate quadrilaterals, we can find the diagonals as:

1. Find polar angles of all points, treating P as origin.
2. Group all points with same polar angles in one equivalence class.
3. Choose the point closest to P in each equivalence class and discard rest of the points.
4. Now, generate line segments from the remaining points. For each polar angle between $[0, \pi)$ radians (called θ), check if we have a point at $\theta + \pi$. These two points make one line segment (note that P lies on this segment).
5. These line segments are treated as diagonals of the quadrilateral.

Runtime Analysis: The triangle case remains the same, taking $O(N^3)$. For quadrilateral, we will generate at most $N/2$ sets of points in $O(N)$ and then we can check all combinations of these segments for possible quadrilateral candidates in $O((N/2)^2)$. So, total runtime will be:

$$O(N^3 + N + (N/2)^2) = O(N^3)$$

Kick Start 2021 - Round F

Analysis: Trash Bins

For every house, we have to find the closest house with a trash bin. This can either be the same house, or some other house to its left or right as at least one house has a trash bin in front of it.

Let $F(i)$ denote distance that the i -th house owner has to walk to take their trashes out. The

final answer is $\sum_{i=1}^N F(i)$.

Test Set 1

For i -th house, we find $F(i)$ by iterating over all the houses and picking the house j such that $S_j = 1$ and the distance between the house i and j is least.

Complexity : $O(N^2)$ per test case

Test Set 2

For i -th house, let $L(i)$ denote the closest house to its left which has trash bin in front of it and $R(i)$ denote the closest house to its right with a trash bin in front of it. We can find $L(i)$ and $R(i)$ for all the houses in one linear pass.

$$L(i) = \begin{cases} -\infty & \text{if } i = 1 \\ i - 1 & \text{if } i > 1 \text{ and } S_{i-1} = 1 \\ L(i - 1) & \text{if } i > 1 \text{ and } S_{i-1} = 0 \end{cases}$$

$$R(i) = \begin{cases} \infty & \text{if } i = N \\ i + 1 & \text{if } i < N \text{ and } S_{i+1} = 1 \\ R(i + 1) & \text{if } i < N \text{ and } S_{i+1} = 0 \end{cases}$$

$$F(i) = \begin{cases} 0 & \text{if } S_i = 1 \\ \min(i - L(i), R(i) - i) & \text{if } S_i = 0 \end{cases}$$

You might have to deal with overflow issues depending on the data types used as the maximum value of answer does not fit 32-bit integer data types.

You can take ∞ as any value $\geq 2 \times N$.

Complexity : $O(N)$ per test case